Migration Path Options Analysis — v2

Cloudinary → UTA S3 (Enterprise External-Storage)

Project: AwsMigration — UTA Cloudinary → S3 Migration **Author:** Edgar (Architect) **Date:** March 20, 2026 **Status:** Updated — post Q&A 3 (Danny / Cloudinary PS Pre-Engagement Call) **Supersedes:** migration-path-options-analysis.md (v1, March 15, 2026) **Distribution:** UTA Stakeholders, Cloudinary Professional Services

---

Purpose

This document presents a structured comparison of the two remaining viable migration paths for moving ~3 TB of binary assets (images, videos, audio, PDFs) from Cloudinary-managed storage to UTA-owned S3, while retaining Cloudinary as the CDN and metadata management layer (BYOB — Bring Your Own Bucket).

Q&A 3 resolved the single most important open question from v1: `cld clone` is now confirmed to copy binary assets and metadata server-side, which collapses the original three-option space into a direct comparison between two credible options. Option C (S3 cross-account copy) remains deferred to the PS scoping session; it is not a planning basis.

Prerequisite (applies to both options): Cloudinary PS must configure the new BYOB-backed environment — and set the S3 backend pointer — **before** any migration work begins. This is a hard dependency confirmed in Q&A 3.

What Q&A 3 Confirmed (Key Inputs to This Analysis)

Question (prev. open)	Answer Confirmed
Does `cld clone` copy binary assets + metadata?	**Yes — server-side, no intermediary machine.** Copies binaries + `public_id`, tags, context fields.
Is the copy server-side or through the client?	**Server-side.** No EC2/bandwidth required for binary transfer.
Does `cld clone` preserve `public_id`, tags, context?	**Yes.** Fully preserved.
Is structured metadata (taxonomy) copied by `cld clone`?	**No.** Structured metadata taxonomy requires a separate bulk export → re-import step.
Are upload presets and env config copied?	**No.** Must be independently reconfigured on the new environment.

Question (prev. open)	Answer Confirmed
Does `cld clone` support incremental / phased execution?	**Yes.** Supports search expressions to clone subsets (by folder, prefix, etc.).
Is there an async mode for large datasets?	**Yes.** Async option available + webhook notifications on completion.
Does `cld sync` support environment-to-environment?	**No.** `cld sync` is local filesystem ↔ Cloudinary only. Eliminated as a migration tool.
When does PS configure BYOB?	**Before the clone runs.** BYOB is a prerequisite for `cld clone` to write to UTA S3.
CDN cache behavior at CNAME cutover	If CNAME is preserved, cached content continues to be served correctly; cache purge may not be needed. (PS to confirm.)
S3 key naming convention under BYOB	**Deferred to PS scoping.** UTA control over key structure is unknown.
Cross-account S3 copy feasibility	**Deferred to PS scoping.** Not a planning basis.

The Two Viable Options

Option A — `cld clone` (Cloudinary-Native Migration)

PS configures the new Cloudinary environment with BYOB pointing to UTA's S3 bucket. A `cld clone` command is then executed against the source environment. Cloudinary performs a **server-side copy** of all binary assets and most metadata directly into the new environment. Because the target environment is BYOB-backed, the binaries land in UTA's S3 bucket automatically — no intermediate machine, no egress routing.

The migration is completed in two additional steps: 1. **Structured metadata taxonomy** is bulk-exported from the source environment and re-imported into the new environment via the Admin API. 2. **Upload presets and environment configuration** are manually reconfigured on the new environment.

`cld clone` supports **search expressions** so the operation can be scoped to folders or subsets for phased execution and validation before committing to the full ~3 TB. An **async flag + webhook** notification is available for long-running operations.

Implementation effort: Low engineering — primarily operational. PS handles BYOB setup; delivery team handles structured metadata export/import scripting and env config replication.

Option B — Worker Fleet (Custom Uploader Loop)

A purpose-built migration engine: a Python controller enumerates all asset `public_ids` from the source environment (via Admin API, respecting the 5,000 req/hr rate limit), then enqueues them into SQS. Stateless workers pull from the queue, download each binary from the source Cloudinary environment (or its CDN), and re-upload it to the new Cloudinary environment via the **Cloudinary Upload API** — which, with BYOB active on the target environment, writes binaries to UTA S3 automatically. Per-asset metadata — tags, context, structured metadata fields — is explicitly written during the upload or via a subsequent Admin API call.

DynamoDB tracks per-asset state (status, attempts, errors, `s3_etag`). CloudWatch and Datadog provide real-time observability. A dead-letter queue captures failures for targeted remediation.

Implementation effort: High engineering — full controller + worker + infra build-out (ECS/Batch, SQS, DynamoDB, CloudWatch). Full programmatic control over every asset.

Comparison Table

Quality Attribute	**A — `cld clone`**	**B — Worker Fleet (Upload API Loop)**
Feasibility (confirmed)	✅ Core behavior confirmed by Q&A 3	✅ Fully understood
Binary Transfer Method	✅ Server-side (Cloudinary-to-Cloudinary → UTA S3) — no intermediary	🟡 Client-mediated — workers download + re-upload via Upload API
Data Egress Cost	✅ Zero — server-side copy, no bandwidth through intermediary	🔴 Incurred — download from source Cloudinary + re-upload to new env
Transfer Speed (~3 TB)	✅ Fast — server-side, no client bandwidth constraint	🟡 Tunable via worker concurrency; bounded by API throughput and bandwidth
`public_id` Preservation	✅ Confirmed preserved by `cld clone`	✅ Guaranteed by design
Tags + Context Metadata	✅ Preserved automatically by `cld clone`	✅ Explicit per-asset write
Structured Metadata Taxonomy	🟡 Separate export → re-import step required (scripted, not automatic)	✅ Explicit per-asset write during migration — no separate step
Upload Presets /	🟡 Must be manually	🟡 Same requirement —

Quality Attribute	**A — `cld clone`**	**B — Worker Fleet (Upload API Loop)**
Env Config	reconfigured on new environment	independent of migration option
Admin API Rate Limit Exposure	🟡 CLI may call Admin API internally; rate behavior at scale not confirmed	✅ Controlled — token-bucket rate limiter, enumeration stays within 5,000 req/hr
Cloudinary PS Dependency	🟡 High — BYOB setup + clone behavior must be validated by PS before execution	🟡 Medium — PS scoped to BYOB setup only; migration engine is self-directed
Operational Visibility	🔴 Low — CLI is a black box; no per-asset state tracking	✅ Full — DynamoDB state, DLQ, CloudWatch, Datadog
Per-Asset Retry / DLQ	🔴 Not available — failure is whole-run or folder-scoped	✅ Native — exponential backoff + SQS DLQ per asset
Phased / Incremental Execution	✅ Yes — search expressions allow folder-scoped clone batches	✅ Native — enqueue any subset of assets
Async Execution	✅ Yes — async flag + webhook notification on completion	✅ Native — worker fleet is inherently async
Pilot Batch (100 assets)	✅ Scope clone to a folder; harder to instrument per-asset	✅ Native — enqueue exactly 100 assets
Rollback Capability	🟡 Moderate — new env isolated; partial-run state unclear without per-asset tracking	✅ Strong — DynamoDB snapshot per asset; explicit rollback path
Risk to Live Environment	✅ Isolated — new Cloudinary env + UTA S3 only	✅ Isolated — workers touch new env only
Engineering Effort	✅ Low — primarily operational + two scripted steps	🔴 High — full infra + application build-out
AWS Infra Required	✅ Minimal — S3 bucket + IAM role for BYOB	🟡 Moderate — ECS/Batch, SQS, DynamoDB, CloudWatch
URL Continuity (CNAME)	✅ Confirmed viable	✅ Confirmed viable

Legend: ✅ Confirmed favorable | 🟡 Uncertain / conditional | 🔴 Risk or blocker

Quality Attribute Summary (Architecture Lens)

Quality Attribute	Recommended Option	Rationale
Lowest Engineering Effort	**A**	Server-side clone + two scripted steps; no infra build-out required
Lowest Data Transfer Cost	**A**	Server-side copy = zero egress; Option B incurs download + upload bandwidth for ~3 TB
Fastest Transfer	**A**	No client bandwidth constraint; Cloudinary handles transfer natively
Reliability / Per-Asset Retry	**B**	DLQ + backoff designed for partial failures at 3 TB scale
Observability	**B**	Only option with per-asset state, dashboards, and alerting
Rollback Safety	**B**	DynamoDB snapshots + isolated environment = safest rollback story
Lowest PS Dependency	**B**	PS scope well-defined; migration engine is entirely self-directed after BYOB setup
Structured Metadata Control	**B**	Explicit per-asset write during migration — no separate taxonomy step required

---

Recommendation

Preferred Path: Option A (`cld clone`) — with Option B as Fallback

Q&A 3 removed the critical blocker for Option A. `cld clone` is now confirmed to perform a server-side binary + metadata copy, preserving `public_id`, tags, and context fields — the majority of UTA's metadata surface area. The structured metadata taxonomy gap is real but manageable: it requires a scripted export/re-import step, not a fundamental re-architecture.

Option A is preferred because: - Zero data egress cost vs. downloading and re-uploading ~3 TB through an intermediary. - Dramatically lower engineering effort — no

SQS/DynamoDB/ECS build-out required. - Cloudinary handles the binary transfer natively; delivery team effort is scoped to structured metadata scripting and env config replication. - Phased execution via search expressions provides the same incremental validation story as Option B. - Async mode + webhooks give sufficient operational awareness for a PS-supervised operation.

Option A's residual risks that must be resolved in the PS scoping session: 1. Admin API rate consumption inside `cld clone` at ~3 TB scale — unknown and potentially throttling. 2. S3 key naming convention under BYOB — UTA control is unknown; affects IAM scoping and bucket organization. 3. Behavior of delivery requests during the empty-bucket window of a rolling clone — fallback to old env? 404? 4. Whether async + webhook gives sufficient failure visibility at this scale, or whether a thin wrapper script is needed.

Option B remains the fallback of last resort. If PS scoping reveals that `cld clone` has unacceptable rate-limit behavior, insufficient error reporting, or cannot be reliably validated at 3 TB, the Worker Fleet provides full programmatic control at the cost of higher engineering effort and egress fees.

Hybrid Consideration

A lightweight hybrid is worth evaluating: use `cld clone` for binary + primary metadata transfer, and pair it with a thin monitoring wrapper (Lambda + DynamoDB) that tracks which folder-scoped clone batches have completed and validates asset counts against the source environment. This captures most of Option A's cost and speed benefits while adding enough observability to reduce operational risk — without building a full SQS/ECS pipeline.

Remaining Open Questions for PS Scoping Session

#	Question	Relevant Option	Impact if Unresolved
1	What Admin API request rate does `cld clone` consume internally per asset at scale? Does it approach or exceed 5,000 req/hr?	A	May throttle mid-migration; could force Option B
2	What is the S3 key/path structure Cloudinary writes under BYOB? Is it stable and documented?	A + B	Required for IAM scoping, bucket organization, and any post-migration S3 tooling

#	Question	Relevant Option	Impact if Unresolved
3	Does UTA have any control over the S3 key naming convention under BYOB?	A + B	Affects long-term S3 organization strategy
4	How does `cld clone` report per-folder or per-asset failures in async mode? Is there a log or API to inspect partial failures?	A	Determines whether a monitoring wrapper is needed
5	How should delivery requests for assets not yet cloned be handled during a rolling transfer? (Fallback to old env? 404?)	A	Cutover sequencing risk
6	Can the Admin API rate limit be temporarily raised to 20,000+ req/hr to support enumeration in Option B if needed?	B	Determines max throughput and migration duration
7	We need clarity on how the Structured Metadata will be moved to the New Environments. This is a step performed by PS and must be considered in the migration plan	A	This will affect the cutover plan
8	What is PS engagement scope, estimated hours, and timeline for BYOB configuration + environment	All	Required for project planning and budgeting

#	Question	Relevant Option	Impact if Unresolved
	setup?		

Assumptions Carried Forward

- A new Cloudinary environment is mandatory before any migration activity (confirmed Q&A 3).
- PS configures BYOB backend before any clone or upload operation runs (confirmed Q&A 3).
- `public_id` values will be preserved exactly in the new environment (confirmed Q&A 2).
- CNAME cutover approach is viable for URL continuity (confirmed Q&A 2).
- `cld sync` is eliminated — local filesystem only, not usable for environment-to-environment transfer (confirmed Q&A 3).
- Old Cloudinary environment is retained post-migration as rollback safeguard (confirmed Q&A 2).
- Structured metadata taxonomy requires a separate export/re-import process regardless of option chosen (confirmed Q&A 3).
- Upload presets and environment configuration must be independently reconfigured on the new environment (confirmed Q&A 3).
- Encryption at rest (SSE-KMS) is mandatory on the UTA S3 bucket; KMS key policy must permit Cloudinary's IAM principal.
- Migration runs are scheduled outside West Coast peak hours (9 AM – 5 PM PT).

Related Documents

- v1: Migration Path Options Analysis
- Cloudinary Q&A 1
- Cloudinary Q&A 2
- Cloudinary Q&A 3